RISHIHOOD UNIVERSITY
School of Technology

B.Tech. Computer Science & Artificial Intelligence
System Design and Software Engineering (CSA234)
Even Semester, 2025-2026

PROJECT REPORT

ShopSmart

A Production-Grade Full-Stack E-Commerce Platform

Submitted By

GitHub ID	Name	Role
Taj-2005	Shaik Tajuddin	Project Founder & Team Lead
pranavi015	Pranavi	SOLID Principles Refactoring
omkar-hadole	Omkar Hadole	Design Patterns & Business Logic
iampraneeth1116	Praneeth	UML Diagrams & Documentation
ashrith-07	Ashrith	Business Logic Implementation

Academic Year: 2025-2026
Submission Date: April 2026

CERTIFICATE

This is to certify that the project report titled "ShopSmart — A Production-Grade Full-Stack E-Commerce Platform" has been prepared and submitted by the following students of the B.Tech. Computer Science & Artificial Intelligence programme at Rishihood University, in partial fulfilment of the requirements for the course System Design and Software Engineering (CSA234), Even Semester, Academic Year 2025-2026.

GitHub ID	Student Name
Taj-2005	Shaik Tajuddin
pranavi015	Pranavi
omkar-hadole	Omkar Hadole
iampraneeth1116	Praneeth
ashrith-07	Ashrith

The work presented in this report is original and has not been submitted elsewhere for the award of any degree or diploma. The project demonstrates the application of software engineering principles, design patterns, UML modelling, and production-grade system development as outlined in the course curriculum.

Acknowledgement

We, the students of B.Tech. Computer Science & Artificial Intelligence at Rishihood University, wish to express our sincere gratitude to all those who contributed to the successful completion of this project.

We are deeply thankful to our course instructor, Deepak, for his invaluable guidance, constructive feedback, and continued encouragement throughout the development of this project. His insights into system design, software engineering principles, and architectural thinking have been instrumental in shaping the quality and structure of ShopSmart.

We extend our appreciation to the Faculty of Technology at Rishihood University for providing an academic environment that fosters practical, hands-on learning and encourages students to bridge the gap between theoretical concepts and real-world engineering challenges.

We also acknowledge the open-source community whose tools and frameworks — including Next.js, Express.js, Prisma, TypeScript, Tailwind CSS, Playwright, and Swagger — formed the technological foundation of this platform. Their freely available documentation and community support were indispensable.

Finally, we thank our peers and families for their moral support and patience throughout the duration of this project.

Shaik Tajuddin | Pranavi | Omkar Hadole | Praneeth | Ashrith

B.Tech. CS & AI — Rishihood University — April 2026

Abstract

ShopSmart is a production-grade, full-stack e-commerce platform developed as a capstone project for the System Design and Software Engineering course (CSA234) at Rishihood University. The platform is built using a modern technology stack comprising Next.js 15 (App Router) for the frontend, Express.js with TypeScript for the backend REST API, Prisma ORM with PostgreSQL for data persistence, and Swagger/OpenAPI 3.0 for API documentation.

The primary objective of ShopSmart is to demonstrate the practical application of software engineering principles and design patterns in the construction of a scalable, maintainable, and secure web application. The codebase rigorously implements all five SOLID principles — Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion — and employs ten design patterns drawn from the classic Gang of Four taxonomy, including Singleton, Observer, Factory, Strategy, Repository, Adapter, Template Method, Chain of Responsibility, Decorator, and Command.

The system provides a three-tier Role-Based Access Control (RBAC) model supporting Customer, Admin, and Super Admin roles. Security is enforced through cookie-based JWT authentication using httpOnly cookies, preventing exposure to cross-site scripting attacks. The platform features a complete customer storefront, a feature-rich administration dashboard, and a super-administration panel for system-level configuration.

ShopSmart is live in production at shopsmart.taj.works, with the backend API accessible at shopsmart-server.taj.works and interactive Swagger documentation at shopsmart-swagger.taj.works/api-docs. The infrastructure includes Docker and Docker Compose for containerisation and optional Terraform modules targeting AWS ECS Fargate, ECR, VPC, and ALB for cloud deployment. End-to-end tests are implemented with Playwright, and unit tests use Jest.

This report documents the architecture, system design decisions, implementation details, deployment strategy, and the application of software engineering best practices throughout the development lifecycle.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Literature Review
6. System Architecture
   - 6.1 High-Level Architecture
   - 6.2 Backend Layered Architecture
   - 6.3 Frontend Architecture
   - 6.4 Request Lifecycle
7. Technology Stack
8. System Design
   - 8.1 UML Diagrams
   - 8.2 Design Patterns Used
   - 8.3 SOLID Principles Implementation
9. Implementation Details
   - 9.1 Frontend
   - 9.2 Backend
   - 9.3 API Design & Swagger Documentation
10. Deployment & DevOps
11. Features & Functionalities
12. Testing & Validation
13. Results & Screenshots
14. Challenges Faced & Solutions
15. Future Enhancements
16. Team Contributions Overview
17. Live Deployments
18. Conclusion
19. References

1. Introduction

E-commerce has become the backbone of global retail, with millions of transactions occurring across digital storefronts every day. Building a production-quality e-commerce platform demands far more than functional code — it requires disciplined software architecture, rigorous application of engineering principles, thorough security design, and a maintainable codebase that can evolve with changing business requirements.

ShopSmart is a fully functional, production-deployed e-commerce platform developed as a group project for the System Design and Software Engineering course at Rishihood University. It serves as both a working commercial application and a showcase of applied software engineering excellence. The platform enables customers to browse and purchase products, administrators to manage the catalogue and operations, and super administrators to configure the entire system.

The project was conceived with a clear engineering mandate: every architectural decision must be deliberate, every abstraction must serve a purpose, and every design pattern must solve a concrete problem present in the codebase. The team deliberately identified existing violations of software design principles in the initial codebase and systematically refactored the architecture to comply with SOLID principles and industry-standard design patterns.

ShopSmart is a monorepo comprising a Next.js 15 frontend, an Express.js TypeScript backend, and an optional Terraform-based AWS infrastructure definition. It is deployed to a custom domain (taj.works), containerised with Docker, and tested with both end-to-end (Playwright) and unit (Jest) test suites. The Swagger documentation at shopsmart-swagger.taj.works/api-docs provides an interactive reference for all API endpoints.

This project report documents the complete journey from problem identification and system design through to production deployment, covering architecture decisions, design pattern implementations, SOLID principle compliance, testing strategy, and future roadmap.

2. Problem Statement

Modern web applications, particularly e-commerce platforms, face a set of recurring engineering challenges that, when left unaddressed, result in systems that are fragile, difficult to test, and expensive to evolve.

The initial codebase of ShopSmart, developed with a focus on rapid feature delivery, exhibited several architectural deficiencies that are common in early-stage software projects. These issues were systematically identified and became the basis for the engineering objectives of this project:

- Route handlers in the original codebase mixed HTTP parsing, business logic, and database queries within the same function, violating the Single Responsibility Principle. Files such as auth.service.ts handled token generation, email sending, and password hashing simultaneously.
- Prisma ORM clients were imported directly in over eight different route files, creating tight coupling between the HTTP layer and the data persistence layer. This made unit testing impossible without a live database connection.
- The addition of new features — such as a new payment gateway, email provider, or discount strategy — required modifying existing, working code rather than extending it through new implementations. This violated the Open/Closed Principle.
- There were no formal interfaces or abstractions separating high-level business logic from low-level infrastructure concerns such as nodemailer or bcryptjs, making the system resistant to change.
- No event-driven mechanism existed for handling side effects (e.g., audit logging, inventory updates, notification dispatch). Every side effect was embedded directly into the order creation route, creating a growing list of direct dependencies.
- Security mechanisms, while functional, lacked formal middleware pipeline ordering and centralised error handling, leading to inconsistent HTTP error responses across different routes.

The project thus addresses both the functional requirements of a production e-commerce platform and the non-functional engineering requirement of building software that is maintainable, testable, extensible, and secure by design.

3. Objectives

The primary objectives of the ShopSmart project, aligned with the learning outcomes of CSA234, are as follows:

1. To design and implement a production-grade, full-stack e-commerce web application using industry-standard technologies including Next.js, Express.js, TypeScript, and Prisma ORM.
2. To systematically apply all five SOLID object-oriented design principles across the backend codebase, providing concrete before-and-after evidence of compliance.
3. To implement a minimum of ten design patterns from the Gang of Four taxonomy — specifically Singleton, Observer, Factory, Template Method, Strategy, Adapter, Repository, Chain of Responsibility, Decorator, and Command — each solving a real problem in the ShopSmart codebase.
4. To design and produce UML artefacts including Entity-Relationship diagrams, Class diagrams, Component diagrams, Deployment diagrams, and Package structure diagrams using PlantUML.
5. To implement a three-tier Role-Based Access Control system with secure cookie-based JWT authentication, enforcing access boundaries between Customer, Admin, and Super Admin roles.
6. To achieve production deployment on a custom domain (taj.works) with separate subdomains for the frontend, backend API, and Swagger documentation.
7. To containerise the application using Docker and Docker Compose, and provide optional AWS infrastructure-as-code using Terraform targeting ECS Fargate.
8. To establish a testing strategy encompassing end-to-end tests with Playwright and unit tests with Jest, ensuring the reliability of critical system flows.
9. To document the entire system comprehensively — including API endpoints via OpenAPI 3.0/Swagger, architecture via UML, and engineering decisions via this report.

4. Scope of the Project

ShopSmart encompasses the following modules and functional areas:

4.1 Customer-Facing Storefront

The customer module provides a complete shopping experience including product discovery, cart management, order placement, order tracking, review submission, profile management, email verification, and password reset functionality.

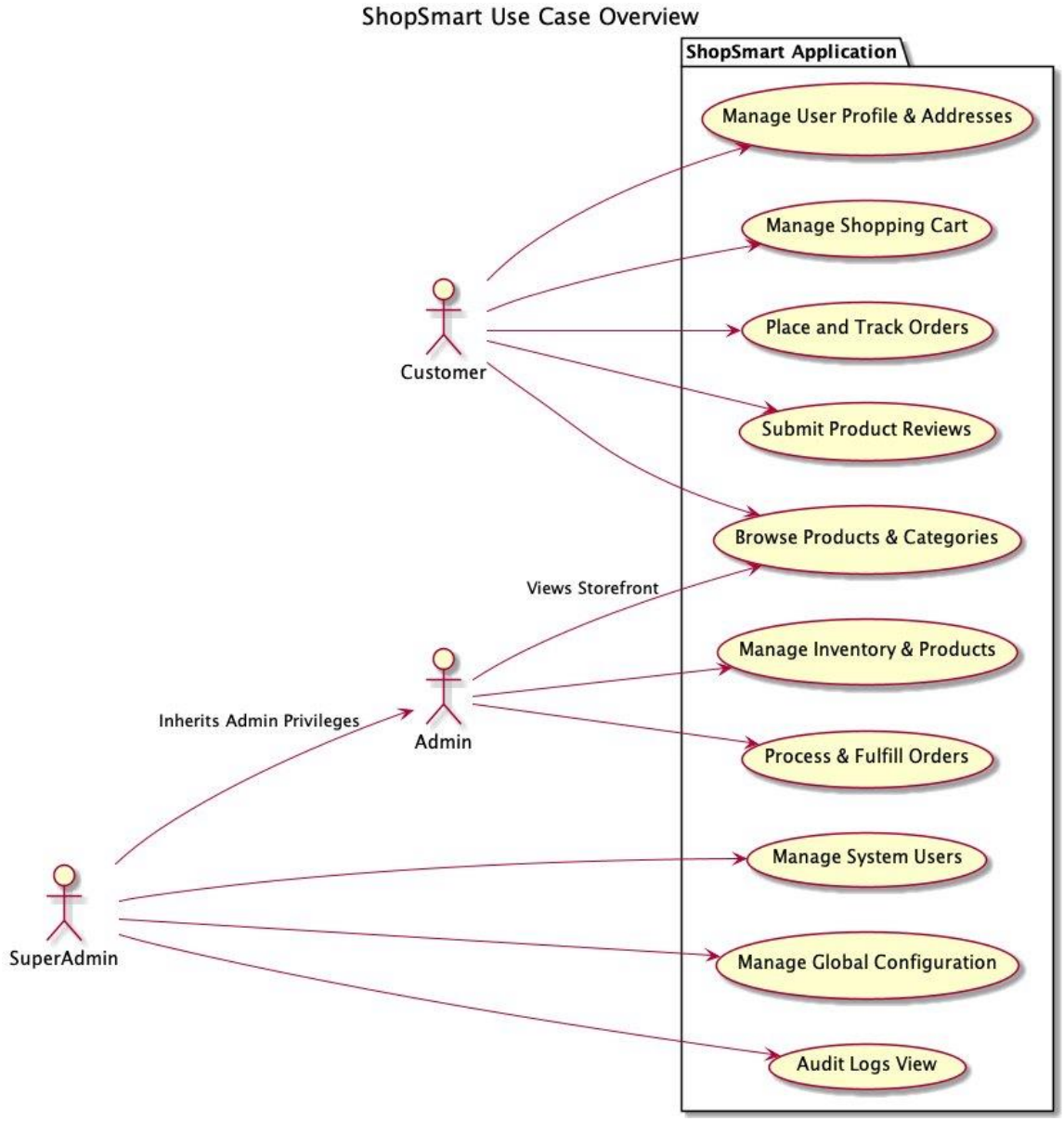

Figure 1: ShopSmart Use Case Overview — Customer, Admin, and Super Admin interactions

4.2 Administration Panel

The admin module provides a feature-complete dashboard for managing the product catalogue, categories, inventory, orders, customer accounts, discount coupons, and moderation of product reviews, along with KPI dashboards and sales reports.

4.3 Super Administration Panel

The super-admin module provides system-level control including admin user management, role assignment, platform-wide configuration, payment gateway settings, shipping provider management, feature flag toggles, and full analytics.

4.4 Engineering Scope

The engineering scope includes: a complete TypeScript backend following a four-layer architecture (Routes, Controllers, Services, Repositories); a dependency injection container (container.ts); a typed event bus with observer registrations; strategy implementations for discount, shipping, token, and email provider; adapter classes for all Prisma-to-DTO transformations; factory classes for error creation and response standardisation; middleware chain of responsibility for authentication and authorisation; a base controller implementing the Template Method pattern; and repository interfaces with Prisma implementations for all domain entities.

4.5 Exclusions

The project does not include a mobile application, real payment gateway integration (payment settings are administrative only), or a recommendation engine. These are identified as future enhancements.

5. Literature Review

The design of ShopSmart draws upon a well-established body of software engineering literature and industry practice.

5.1 Design Patterns

The foundational reference for the ten design patterns implemented in ShopSmart is the seminal work Design Patterns: Elements of Reusable Object-Oriented Software by Gamma, Helm, Johnson, and Vlissides (the Gang of Four, 1994). This work introduced the formal classification of creational, structural, and behavioural patterns and remains the primary reference for pattern-based architecture. Head First Design Patterns (Freeman and Robson, O'Reilly) provides a pedagogically accessible treatment of the same patterns with practical code examples.

5.2 SOLID Principles

The SOLID principles were articulated by Robert C. Martin (Uncle Bob) and are comprehensively documented in Clean Architecture (Martin, Prentice Hall, 2017) and Agile Software Development, Principles, Patterns, and Practices (Martin, 2002). The Single Responsibility Principle and Open/Closed Principle were originally described by Bertrand Meyer (Object-Oriented Software Construction, 1988). The Liskov Substitution Principle was formalised by Barbara Liskov in her 1987 keynote.

5.3 Repository and Dependency Injection

The Repository pattern is documented in Patterns of Enterprise Application Architecture (Fowler, Addison-Wesley, 2002). Dependency Injection as an architectural technique is thoroughly covered in Clean Architecture and is practically implemented in frameworks like Spring (Java) and leveraged in Node.js through manual constructor injection, as employed in ShopSmart's container.ts.

5.4 E-Commerce System Architecture

Designing Data-Intensive Applications (Kleppmann, O'Reilly, 2017) informed decisions around stateless API design, horizontal scalability, and database schema evolution. Site Reliability Engineering (Google SRE Team, O'Reilly) influenced the platform's observability approach, including structured logging, health check endpoints, and audit log observer design.

5.5 TypeScript and Modern Web Frameworks

The official documentation for TypeScript, Next.js 15 App Router, Express.js 4, Prisma ORM, and Swagger/OpenAPI 3.0 served as primary technical references throughout development. The Next.js App Router model informed the frontend routing strategy and server component considerations.

6. System Architecture

ShopSmart adopts a monorepo architecture that cleanly separates the frontend application from the backend REST API while sharing TypeScript type definitions and configuration standards. The overall system consists of three deployed services: the Next.js frontend, the Express.js backend API, and the Swagger documentation server, each hosted on a dedicated subdomain under taj.works.

6.1 High-Level Architecture

The system follows a classic three-tier architecture: the presentation tier (Next.js frontend), the application logic tier (Express.js REST API), and the data tier (PostgreSQL database accessed via Prisma ORM). Communication between the frontend and backend uses HTTPS-only REST API calls with credentials (cookies) transmitted automatically by the browser.

The monorepo root contains the client/ directory (Next.js), the server/ directory (Express/Prisma), the e2e/ directory (Playwright tests), the docs/ directory (UML diagrams), and the terraform/ directory (AWS IaC). A docker-compose.yml at the root orchestrates both services for local development.

6.2 Backend Layered Architecture

The backend follows a strict four-layer architecture within each feature module. This layering enforces the Single Responsibility Principle and enables independent testing of each layer.

Layer	File Pattern	Responsibility
Routes	*.routes.ts	Express router wiring, middleware composition, HTTP verb mapping
Controllers	*.controller.ts	HTTP orchestration, response status codes, delegation to services
Services	*.service.ts	Domain business logic, event emission, service composition
Repositories	Prisma*Repository.ts	Data persistence, Prisma query abstraction behind interfaces

Cross-cutting concerns are handled by dedicated directories: interfaces/ holds all contract definitions; factories/ contains factory classes for errors, responses, and services; strategies/ contains swappable algorithm implementations; adapters/ contains DTO transformation classes; events/ contains the EventBus and observer registrations; and middleware/ contains all Express middleware functions.

6.3 Frontend Architecture

The frontend leverages Next.js 15's App Router for file-system-based routing with support for Server Components, Client Components, layouts, and loading states. The architecture is organised around three role-based application sections: the customer storefront under /shop, the administration panel under /admin, and the super-administration panel under /super-admin.

The context/ directory holds the AuthProvider and associated hooks (useIsAdmin, useIsSuperAdmin). The api/ directory contains a shared Axios client configured with withCredentials: true and a refresh-on-401 interceptor. Route protection is enforced through ProtectedRoute, AdminRoute, and SuperAdminRoute wrapper components.

6.4 Request Lifecycle

Every incoming HTTP request passes through the following ordered middleware pipeline before reaching a route handler. This chain of responsibility ensures consistent security enforcement, context enrichment, and error handling:

- requestContextMiddleware: Attaches a unique requestId (UUID v4) and a startTime timestamp to each request for tracing and performance monitoring.
- helmet(): Sets security-relevant HTTP response headers, mitigating common web vulnerabilities.
- cors(): Validates the request origin against the configured FRONTEND_URL, enabling credentials for cross-origin cookie transmission.
- express.json() and cookieParser(): Parse the JSON request body and extract HTTP cookies, populating req.body and req.cookies respectively.
- authenticate(): Reads the accessToken from req.cookies, verifies the JWT signature, and attaches the decoded user payload to req.user.
- authorize(roles): Enforces role-based access control by verifying that req.user.roleType matches the required role(s).
- validate(schema): Applies Zod or express-validator schemas to the request body, returning a 400 error for invalid input before reaching the controller.
- Route Handler / Controller: Executes the business logic by calling the appropriate service method and returning a standardised JSON response via ApiResponseFactory.
- errorHandler: A final Express error-handling middleware that converts all AppError subclass instances into structured JSON responses with appropriate HTTP status codes.

7. Technology Stack

7.1 Frontend Technologies

Technology	Version	Purpose
Next.js	15 (App Router)	React-based framework with SSR/SSG/ISR support and file-system routing
React	19	Declarative UI component model
TypeScript	Strict	End-to-end static type safety
Tailwind CSS	v4	Utility-first CSS framework for responsive UI design
Axios	Latest	HTTP client with interceptor support for token refresh
Framer Motion	Latest	Declarative animations for page transitions and component reveals
Recharts	Latest	Composable charting library for admin and analytics dashboards

7.2 Backend Technologies

Technology	Version	Purpose
Node.js	18+	JavaScript runtime for server-side execution
Express.js	4.x	Minimal, flexible REST API framework
TypeScript	Strict	Type-safe backend development
Prisma ORM	5.x	Type-safe database access layer with migration management
PostgreSQL	15+	Primary relational database
JWT (jsonwebtoken)	9.x	Stateless authentication token generation and verification
bcryptjs	Latest	Password hashing with salt rounds
cookie-parser	Latest	HTTP cookie parsing middleware
Helmet	Latest	Security HTTP header configuration
express-validator	Latest	Request input validation and sanitisation
Swagger UI Express	Latest	Serving interactive OpenAPI 3.0 documentation
Nodemailer	Latest	Email delivery for verification and password reset

7.3 DevOps and Infrastructure

Tool	Purpose
Docker	Application containerisation
Docker Compose	Local multi-service orchestration (frontend + backend)
Terraform	Infrastructure as Code for AWS ECS Fargate, VPC, ALB, ECR, S3
GitHub Actions	CI/CD pipeline for automated build, test, and deployment
Playwright	End-to-end browser automation testing
Jest	Unit testing framework for backend services
PlantUML	UML diagram generation (ER, Class, Component, Deployment)

8. System Design

8.1 UML Diagrams

The system is comprehensively documented through five UML artefacts, developed by Praneeth using PlantUML and stored in the docs/uml/ directory of the repository.

8.1.1 Entity-Relationship Diagram

The ER diagram captures the relational schema of the PostgreSQL database. The core entities are User, Role, Product, Category, Cart, CartItem, Order, OrderItem, Review, Coupon, AuditLog, SystemConfig, and PaymentGateway. Key relationships include: a User belongs to one Role (CUSTOMER, ADMIN, SUPER_ADMIN); a Product belongs to one Category; an Order belongs to one User and contains multiple OrderItems, each referencing a Product; a Cart belongs to one User and contains CartItems; a Review belongs to both a User and a Product.

All primary keys use UUID strings for global uniqueness and to avoid enumerable IDs in API paths. Soft deletion is implemented on User and Product entities via a deletedAt timestamp column.

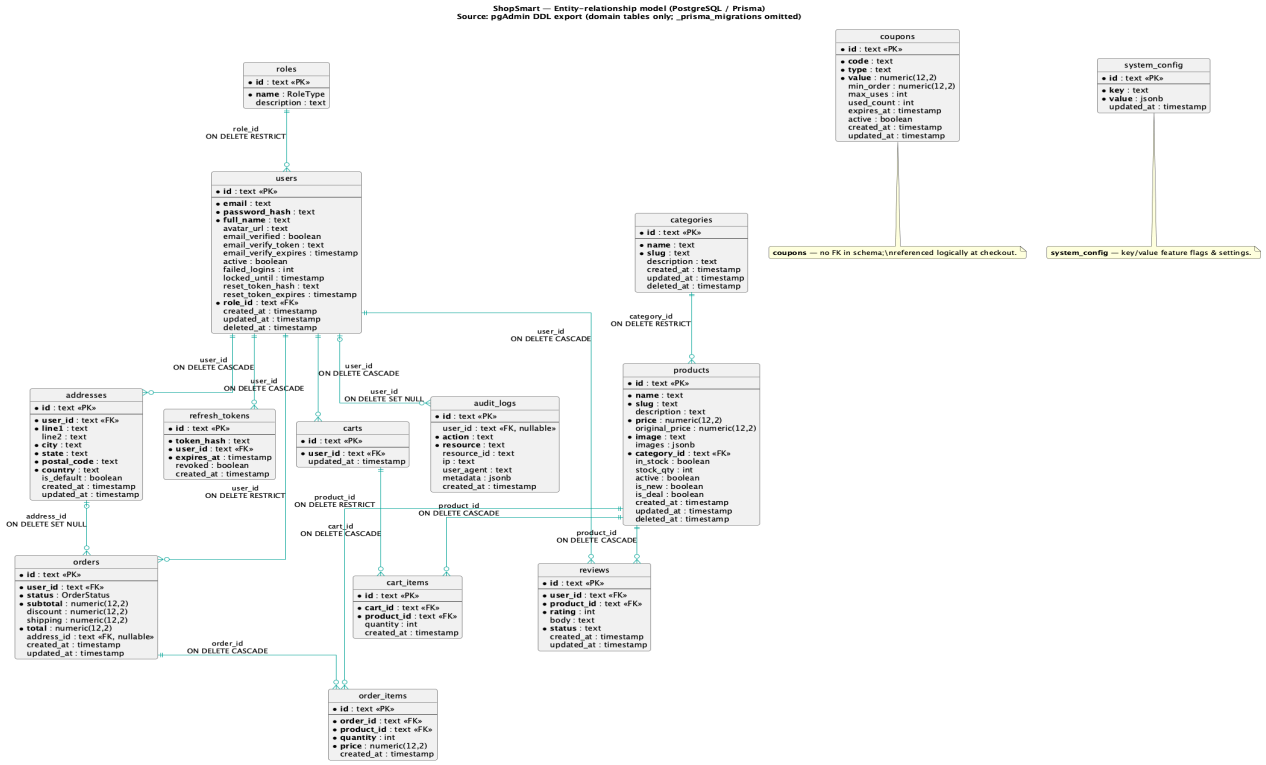

Figure 2: ShopSmart Entity-Relationship Diagram (PlantUML)

8.1.2 Class Diagram (Backend Services)

The class diagram illustrates the service layer hierarchy and dependency relationships. The central classes include AuthService, OrderService, ProductService, UserService, AdminService, and SuperAdminService. Each service class depends on one or more repository interfaces (IUserRepository, IOrderRepository, IProductRepository), injected through constructor parameters. The diagram shows that service classes depend on abstractions (interfaces) rather than concrete Prisma repositories, demonstrating the Dependency Inversion Principle.

8.1.3 Component / System Architecture Diagram

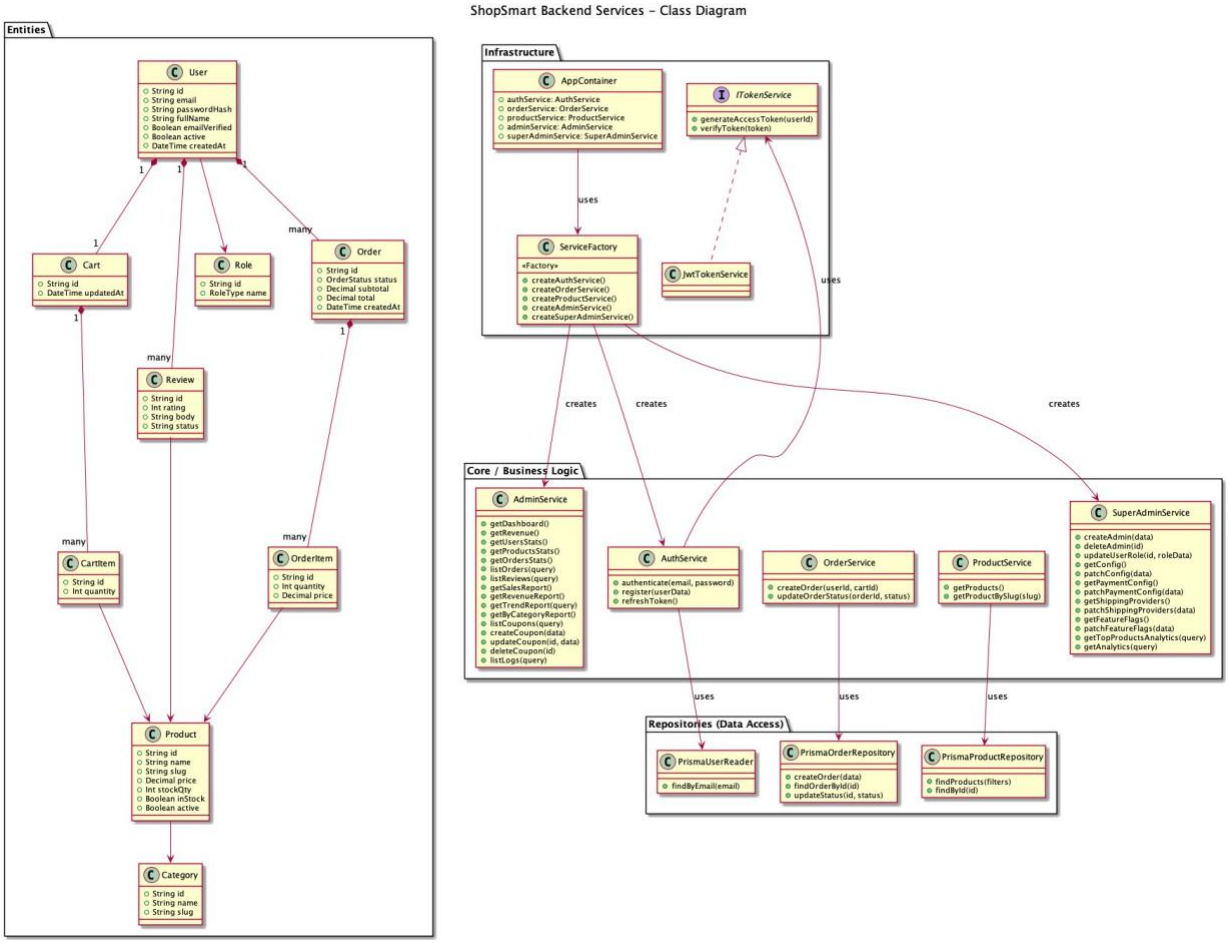

Figure 3: Backend Services Class Diagram — Entities, Infrastructure, Core Logic, Repositories

The component diagram depicts the four main system components — Next.js Client, Express API Server, PostgreSQL Database, and the EventBus — and their interaction contracts. The API Server is further decomposed into its layers (Routes, Controllers, Services, Repositories) to show the internal component boundaries. External integrations shown include the Nodemailer SMTP connection and the optional Stripe payment gateway adapter.

8.1.4 Deployment Diagram

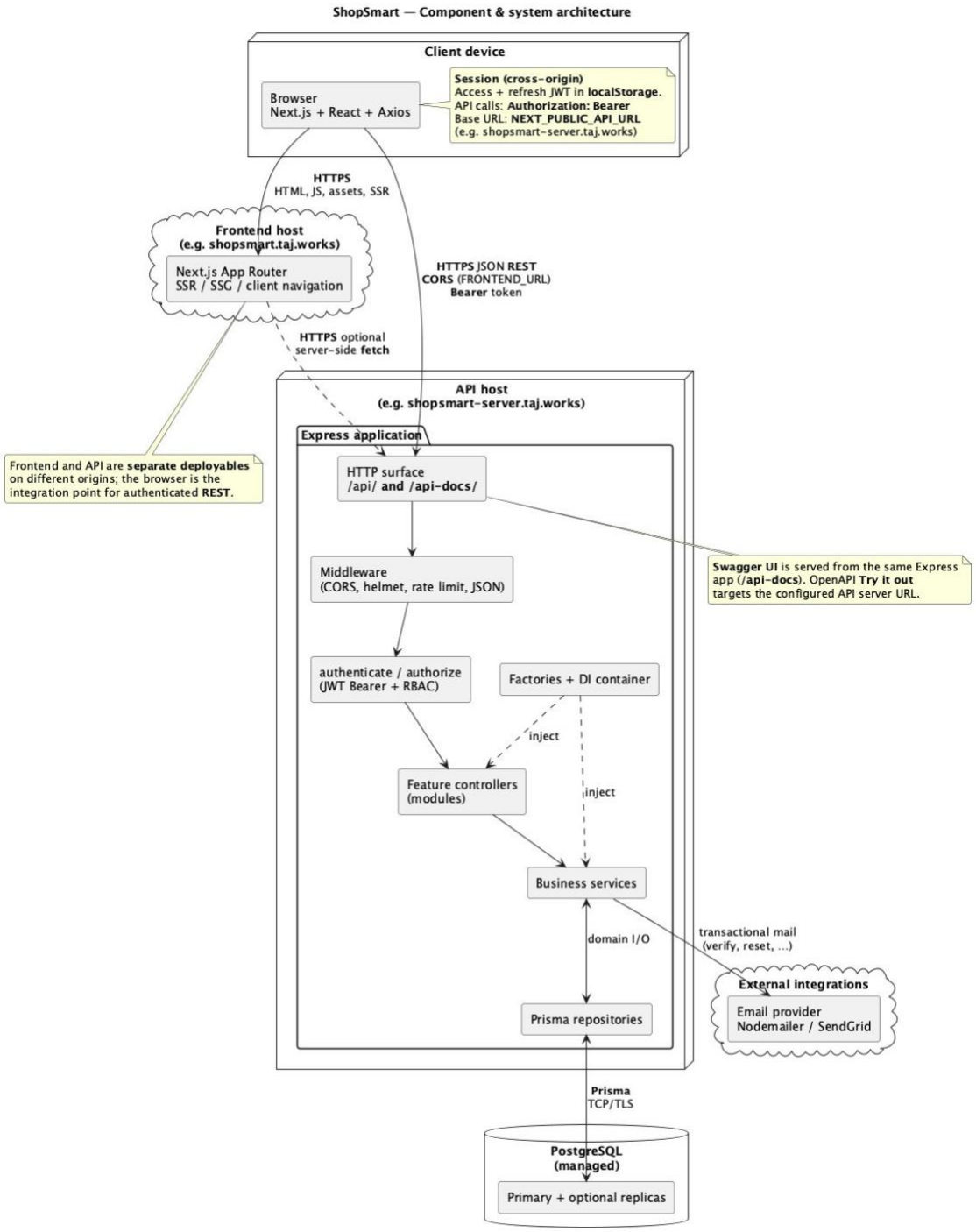

Figure 4: Component & System Architecture Diagram

The deployment diagram represents the production topology on the taj.works domain. The Next.js frontend is deployed to a Node.js server, the Express API to a separate Node.js server, and Swagger UI to a third server, all behind an Application Load Balancer (ALB). PostgreSQL is hosted on a managed database instance. The architecture supports horizontal scaling of the stateless API and frontend layers.

8.1.5 Package Structure Diagram

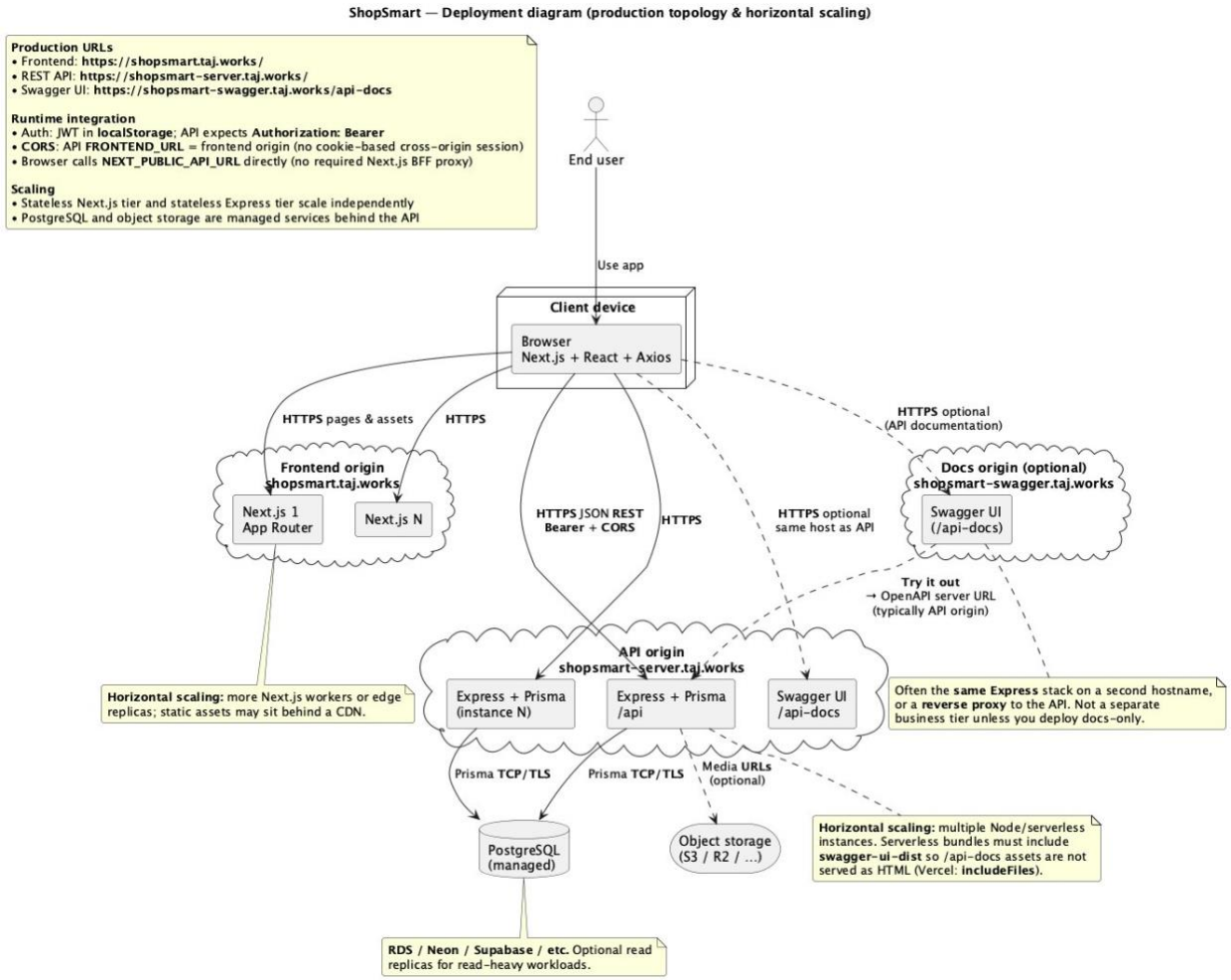

Figure 5: Production Deployment Diagram — taj.works Topology & Horizontal Scaling

The server package structure diagram visualises the directory hierarchy of the backend, showing the modular organisation under src/ with packages for modules/, interfaces/, repositories/, factories/, strategies/, adapters/, commands/, events/, services/, base/, config/, and middleware/.

8.2 Design Patterns Used

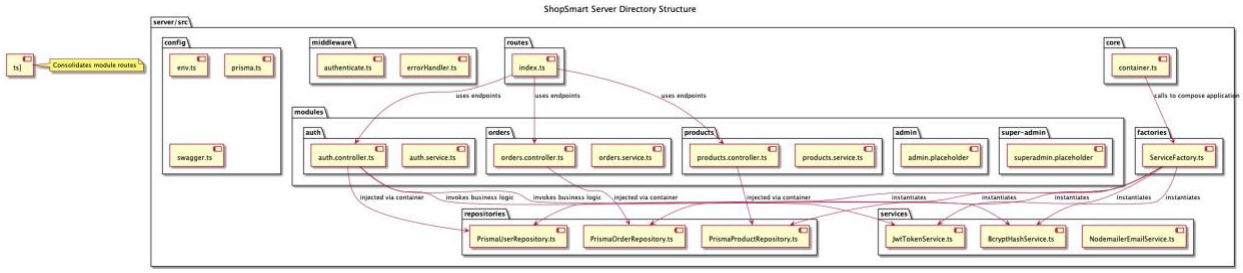

Figure 6: ShopSmart Server Package / Directory Structure

ShopSmart implements ten design patterns, each selected to address a specific, concrete problem in the codebase.

8.2.1 Singleton Pattern (Creational)

Problem Addressed: Multiple instantiations of the Prisma database client, logger, and email transporter across the application would waste resources and cause inconsistent state. Solution: Four Singleton classes ensure exactly one instance exists throughout the application lifecycle: DatabaseService (wrapping PrismaClient with connect/disconnect lifecycle management), LoggerService (structured JSON logger with log level control), ConfigService (validates all required environment variables at startup, throwing on missing values), and EmailTransporterService (single Nodemailer transporter instance). All expose a static getInstance() method with a null-check guard.

8.2.2 Observer Pattern (Behavioural)

Problem Addressed: Side effects such as audit logging, email notification dispatch, and inventory decrements were directly embedded in the order creation route handler. Adding any new side effect required modifying existing, tested code. Solution: A type-safe EventBus singleton implements a publish-subscribe mechanism. Domain events (OrderCreatedEvent, OrderStatusChangedEvent, UserRegisteredEvent, ReviewSubmittedEvent) are emitted by services. AuditLogObserver, NotificationObserver, and InventoryObserver subscribe to relevant events and handle side effects independently. Errors in observer handlers are caught and logged but never propagate to the main request flow.

8.2.3 Factory Pattern (Creational)

Problem Addressed: Calls to new AppError(404, 'Not found', 'NOT_FOUND') were scattered across thirty or more files with no consistency guarantees. Response shapes varied between routes. Solution: Three factory classes centralise creation. AppErrorFactory provides static methods (notFound, conflict, unauthorized, forbidden, validation, internal) that produce correctly configured AppError instances. ApiResponseFactory produces standardised response envelopes ({ success, data, message, meta }). ServiceFactory wires all service classes with their injected dependencies, acting as a composition root.

8.2.4 Template Method Pattern (Behavioural)

Problem Addressed: Every controller action followed the same algorithm — call service, send response, catch error — but this pattern was reimplemented repeatedly in each route handler. Solution: BaseController defines a handleRequest() template method that encapsulates the fixed algorithm. Concrete controller classes (AuthController, OrderController, ProductController, etc.) extend BaseController and call handleRequest(), handleCreated(), or handleMessage() with a handler lambda. The base class handles error propagation to the errorHandler middleware uniformly. Similarly, BaseEmailTemplate defines a render() template method with abstract steps (getSubject, getGreeting, getBody) filled in by concrete email template subclasses.

8.2.5 Strategy Pattern (Behavioural)

Problem Addressed: Pricing calculations, discount logic, and authentication token operations were hardcoded in route handlers. Adding new discount types or token providers required modifying working code. Solution: Multiple strategy families are defined. IDiscountStrategy is implemented by PercentDiscountStrategy, FixedDiscountStrategy, and NoDiscountStrategy. DiscountStrategyFactory selects the appropriate strategy based on the coupon type at runtime. IShippingStrategy is implemented by FreeShippingStrategy, FlatRateShippingStrategy, and FreeAboveThresholdStrategy. ITokenStrategy is implemented by JwtTokenStrategy, with OAuthJwtTokenService available for future OAuth support. IEmailProviderStrategy is implemented

by NodemailerStrategy, with a SendGridStrategy stub for future extension. Runtime strategy selection is controlled by environment variables (AUTH_PROVIDER, EMAIL_PROVIDER).

8.2.6 Adapter Pattern (Structural)

Problem Addressed: Prisma model objects contain Decimal types, raw date objects, sensitive fields (passwordHash, resetTokenHash), and embedded relations that should never be exposed directly in HTTP responses. Solution: Adapter classes perform all Prisma-to-DTO transformations. UserAdapter.toDTO() converts a Prisma User with its Role relation to a UserDTO that excludes all security-sensitive fields. ProductAdapter.toDTO() converts Decimal price fields to JavaScript numbers. OrderAdapter.toDTO() converts an Order with nested OrderItems and Products to a fully nested OrderDTO. StripeAdapter and NodemailerAdapter adapt external library APIs to internal domain interfaces.

8.2.7 Repository Pattern (Structural)

Problem Addressed: Prisma was imported directly in nine different route and service files, creating pervasive coupling between business logic and database access. Unit tests required a live database connection. Solution: Repository interfaces (IUserRepository, IProductRepository, IOrderRepository, ICartRepository, IReviewRepository, ICouponRepository, IAuditLogRepository) define the data access contracts. Prisma implementations (e.g., PrismaUserRepository) encapsulate all Prisma query logic. Services receive repository instances through constructor injection and never import PrismaClient directly.

8.2.8 Chain of Responsibility Pattern (Behavioural)

Problem Addressed: Middleware registration order was inconsistent across routes, creating potential security vulnerabilities. Solution: A standardised, documented middleware chain order is enforced: requestContext, helmet, rateLimit, express.json, cookieParser, authenticate, authorize, validate, handler, errorHandler. The requestContextMiddleware attaches a unique requestId and startTime to every request. The errorHandler must always be the last app.use() registration, capturing all errors regardless of where they originate in the chain.

8.2.9 Decorator Pattern (Structural)

Problem Addressed: Logging, caching, and performance tracking needed to be added to service methods without modifying the service classes themselves. Solution: TypeScript method decorator factories are implemented in src/decorators/. @WithLogging(label) wraps any service method to log start time, end time, duration, and any thrown errors using LoggerService. @WithCache(ttl) caches return values for read-heavy operations. Decorators are applied declaratively at the service class level and are transparent to callers.

8.2.10 Command Pattern (Behavioural)

Problem Addressed: Order mutations and admin actions needed to automatically produce audit trail entries without duplicating logging logic in every service method. Solution: The ICommand interface defines execute() and getAuditEntry() methods. CreateOrderCommand and UpdateOrderStatusCommand encapsulate the operation and its audit metadata. A CommandBus executes the command and writes the audit log entry atomically, ensuring every command is audited without any service needing to directly interact with the AuditLog repository.

8.3 SOLID Principles Implementation

8.3.1 Single Responsibility Principle

Each class and file in the refactored codebase has exactly one reason to change. AuthService orchestrates authentication flows but delegates token generation to ITokenService, password hashing to IHashService, email delivery to IEmailService, and data persistence to IUserRepository. The email.ts file no longer contains business logic — NodemailerEmailService handles only SMTP transport. TokenService handles only JWT operations. The litmus test applied: 'If the email template changes, only EmailService changes; if the JWT algorithm changes, only JwtTokenService changes.'

8.3.2 Open/Closed Principle

New behaviours are added through new implementations of existing interfaces, not by modifying existing classes. Adding SMS notifications requires only creating a new SmsNotificationService implementing INotificationService and wiring it in container.ts — AuthService is not touched. Adding a new discount type requires a new strategy class implementing IDiscountStrategy. The interface definitions in src/interfaces/ are the closed, stable contracts; new features extend the system without modifying them.

8.3.3 Liskov Substitution Principle

A typed error hierarchy (AppError, AuthError, ValidationError, NotFoundError, ConflictError) ensures that the errorHandler middleware handles all AppError subclasses uniformly without instanceof checks beyond the top-level AppError guard. Any class implementing IEmailService, ITokenService, or IDiscountStrategy can be substituted for any other implementation — the calling service does not notice the change. Controller subclasses are fully substitutable via the BaseController contract.

8.3.4 Interface Segregation Principle

Instead of a single fat IUserService interface with fifteen methods that every consumer would depend on, focused interfaces are defined for specific consumer roles: IUserReader (findById, findByEmail) for read-only consumers, IUserWriter (create, update, softDelete) for mutation consumers, ITokenService (signAccess, signRefresh, verify) for token operations only, IHashService (hash, compare, generateRandom) for hashing, and IRoleChecker (hasRole, isAdmin, isSuperAdmin) for authorisation checks.

8.3.5 Dependency Inversion Principle

High-level business logic modules never import concrete infrastructure classes. AuthService does not import PrismaClient, nodemailer, bcryptjs, or jsonwebtoken. All dependencies are declared as constructor parameters typed against interfaces (IUserRepository, IEmailService, ITokenService, IHashService). The single location where concrete classes are instantiated and wired together is src/container.ts (the composition root), making the codebase fully testable with mocks and fakes injected in unit tests.

9. Implementation Details

9.1 Frontend Implementation

The frontend is built with Next.js 15's App Router, providing a structured, file-system-based routing model with support for layouts, loading states, and React Server Components where applicable.

9.1.1 Application Structure

The app/ directory is organised into three role-based sections. The customer storefront under app/shop/ provides product browsing, cart management, and order flows. The administration panel under app/admin/ provides KPI dashboards, product and order management, and coupon administration. The super-administration panel under app/super-admin/ provides system configuration, admin user management, and analytics.

9.1.2 Authentication and Route Protection

Authentication state is managed globally through an AuthProvider in context/auth-context.tsx, exposing useIsAdmin() and useIsSuperAdmin() hooks. Route protection components (ProtectedRoute, AdminRoute, SuperAdminRoute) wrap page components and redirect unauthenticated or unauthorised users to the login page. The Axios client in api/axios.ts is configured with withCredentials: true to automatically transmit HttpOnly cookies on every request.

A response interceptor on the Axios client intercepts 401 Unauthorized responses, transparently calls POST /api/auth/refresh to obtain a new access token, and retries the original failed request once. If the refresh also fails, the user is redirected to /login and the auth context is cleared.

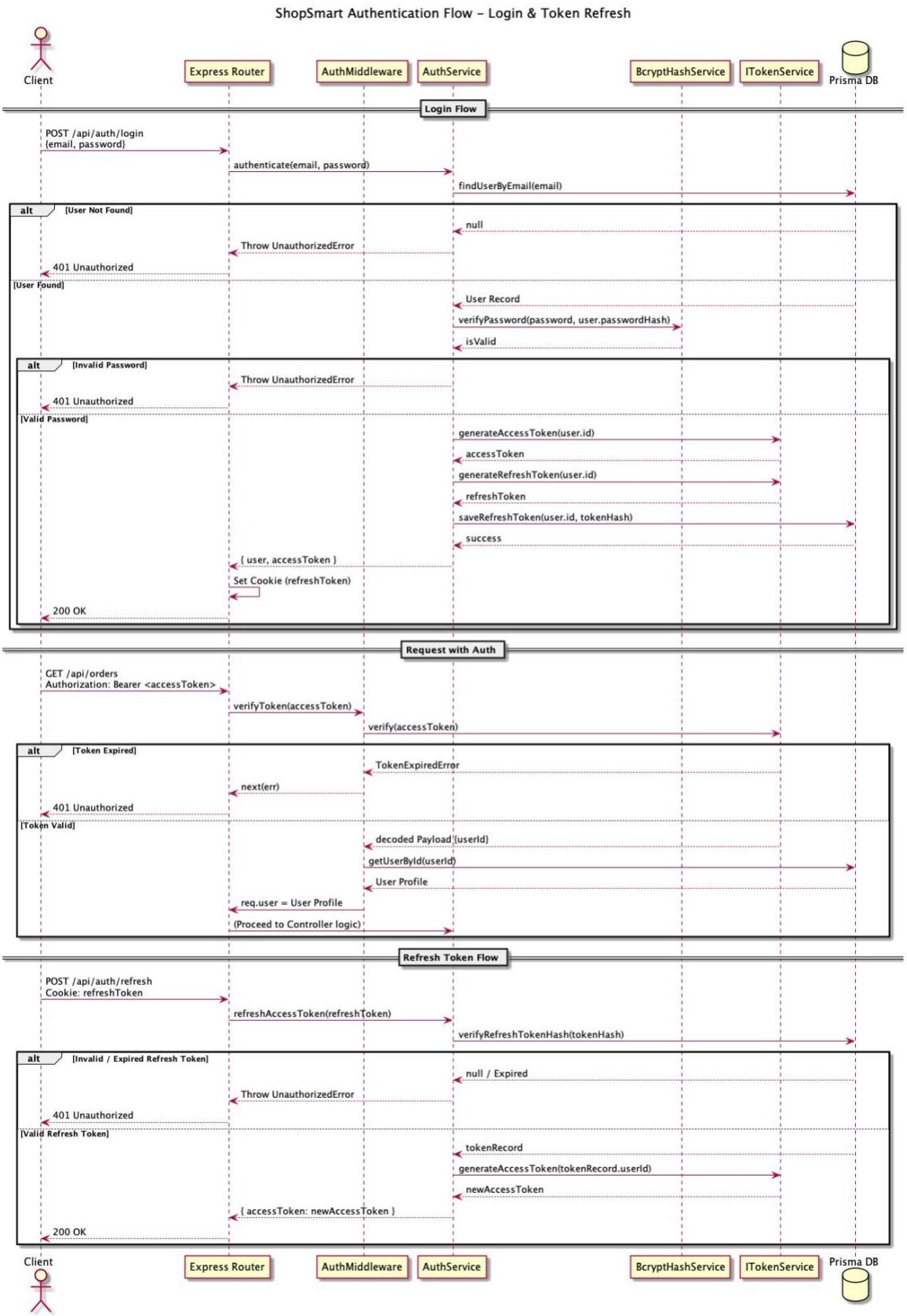

Figure 7: Authentication Flow — Login, Request with Auth, and Token Refresh Sequence

9.1.3 UI Component Architecture

Shared UI primitives are located in components/ui/ (Button, Logo, Input, etc.). Domain-specific components are organised by feature: components/shop/ for product cards, filters, and grids; components/admin/ for dashboard charts, data tables, and sidebar; components/super-admin/ for system configuration panels. Framer Motion is used for page transition animations. Recharts renders the sales, revenue, and user analytics charts in the admin and super-admin dashboards.

9.2 Backend Implementation

9.2.1 Module Structure

Each domain feature is encapsulated in a module under server/src/modules/. The modules are: auth (registration, login, token refresh, logout, email verification, password reset), user (current user profile), users (user CRUD for self and admin), products (catalogue management), categories, cart, orders (checkout and lifecycle), reviews (submission and moderation), admin (dashboard, reports, coupons), and super-admin (admin management, system config, feature flags, analytics).

9.2.2 Dependency Injection Container

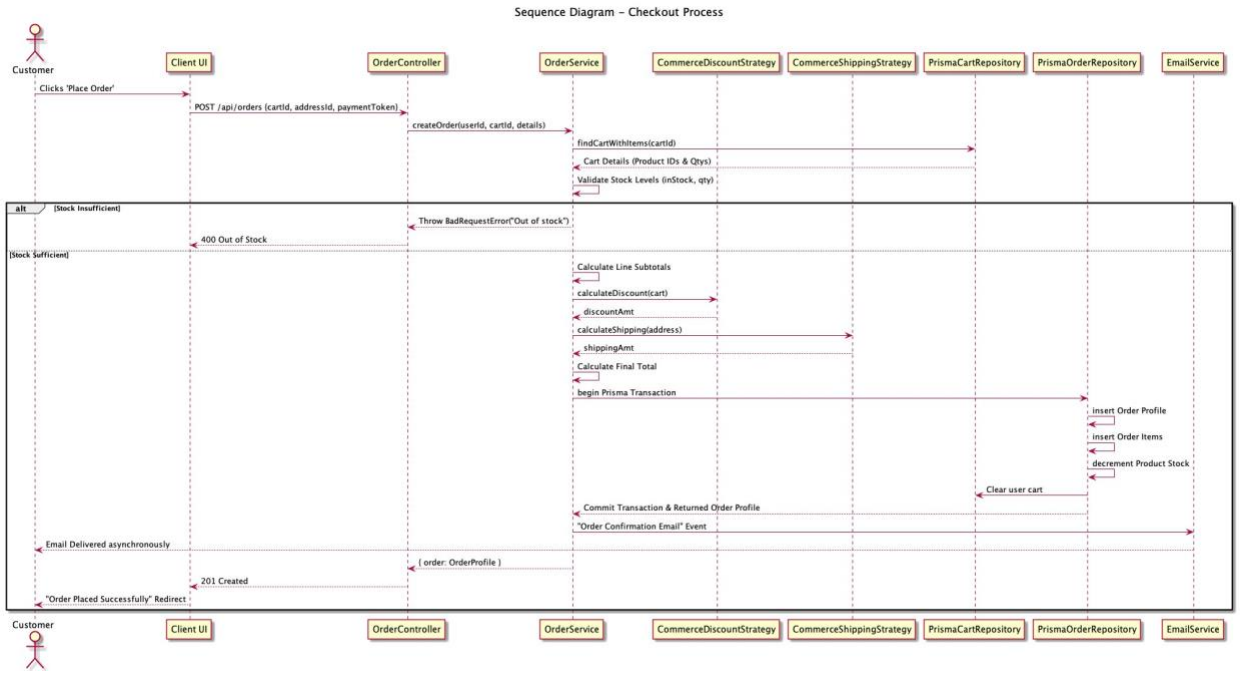

Figure 10: Checkout Process Sequence Diagram — OrderService, Strategies, Repositories

The file server/src/container.ts serves as the composition root for the entire backend. It instantiates all concrete classes (repository implementations, service implementations, strategy implementations) and wires their dependencies by passing them as constructor arguments. This file is the only location in the codebase where new Operator is called on concrete infrastructure classes. All other modules receive their dependencies through interfaces.

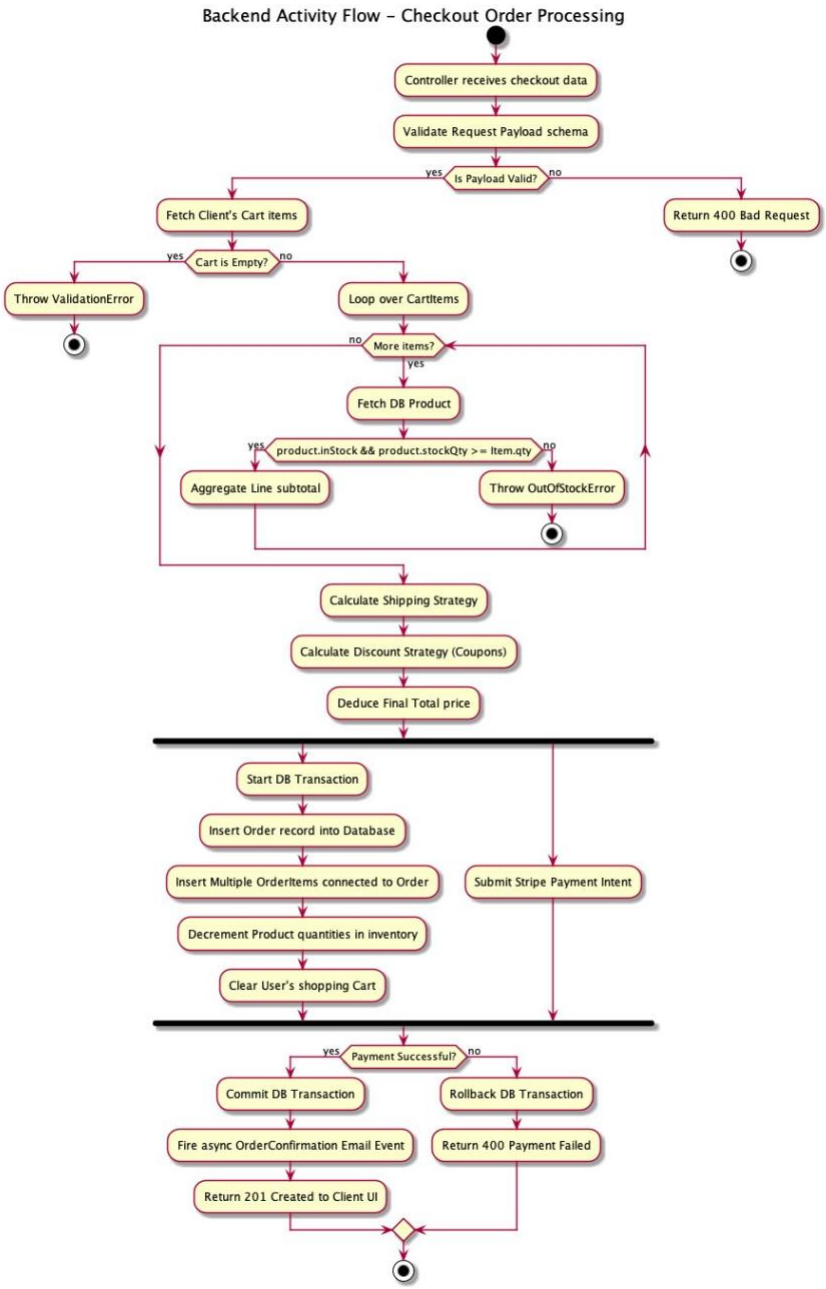

Figure 11: Backend Activity Flow — Checkout Order Processing

The container also registers all EventBus observers at startup. AuditLogObserver, NotificationObserver, and InventoryObserver are instantiated with their dependencies and their register() methods are called, subscribing them to the relevant domain events before the server begins accepting requests.

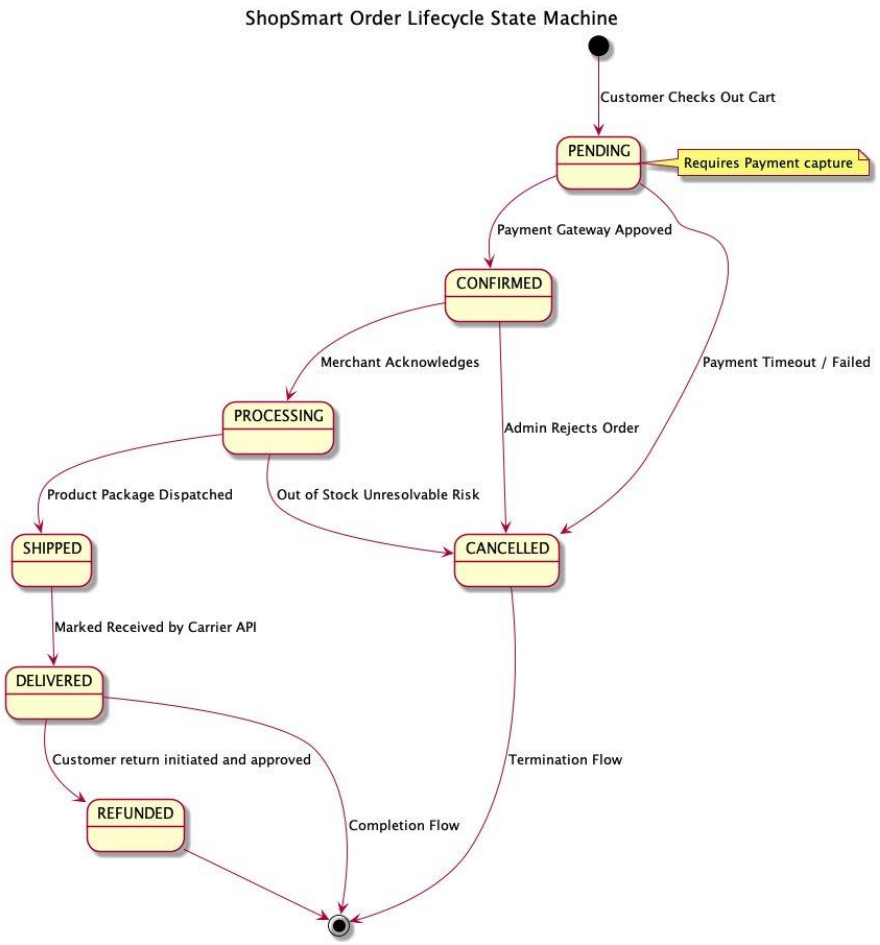

Figure 8: Order Lifecycle State Machine — PENDING through REFUNDED/DELIVERED

The Prisma schema defines the following primary models: User (with role, profile, authentication tokens, and lock state), Role, Product (with soft delete, stock management, and category relation), Category, Cart and CartItem, Order and OrderItem, Review, Coupon, AuditLog, SystemConfig, and PaymentGateway. All models use UUID string primary keys. Prisma migrations manage schema evolution, and prisma migrate deploy is run as part of the production deployment startup command.

9.2.3 Database Schema

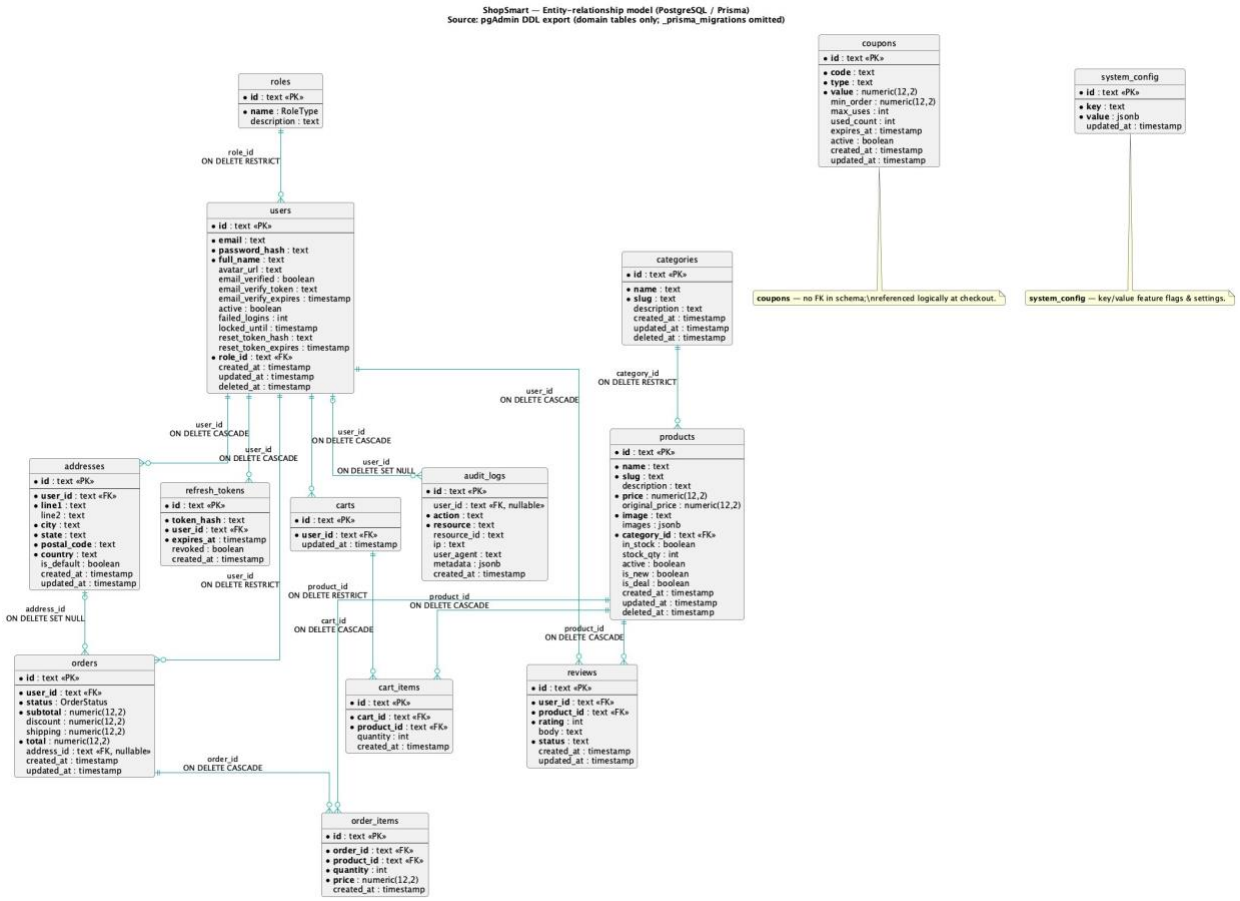

Figure 9: Database Schema — pgAdmin DDL Export (PostgreSQL / Prisma)

9.3 API Design and Swagger Documentation

The REST API follows RESTful conventions with versioned endpoints under the /api prefix. All responses conform to the standardised envelope produced by ApiResponseFactory: { success: boolean, data?: T, message?: string, meta?: { total, page, limit } }. All errors conform to the AppError hierarchy and are serialised by the errorHandler middleware.

Prefix	Module	Auth Required
/api/auth	Register, login, refresh, logout, verify email, forgot/reset password	Public / Authenticated
/api/user	Current user profile (me), update profile, avatar	Authenticated
/api/users	User CRUD, user orders, user cart (self or admin)	Authenticated
/api/products	Product catalogue, search, analytics, CRUD	Public / Admin
/api/categories	Category listing and management	Public / Admin
/api/cart	Cart CRUD, add/update/remove items	Authenticated
/api/orders	Checkout, order history, status updates	Authenticated
/api/reviews	Create, moderate, list reviews	Public / Admin
/api/admin	KPIs, reports, coupons, moderation, audit logs	Admin+
/api/super-admin	Admin CRUD, feature flags, config, analytics	Super Admin
/api/health	System health check	Public

The Swagger documentation is generated using swagger-jsdoc and swagger-ui-express, served from a dedicated subdomain at shopsmart-swagger.taj.works/api-docs. All endpoints are annotated with OpenAPI 3.0 JSDoc comments defining request schemas, response schemas, authentication requirements, and error codes.

10. Deployment & DevOps

10.1 Containerisation

Both the frontend and backend are containerised using Docker. Individual Dockerfiles for the client and server produce optimised production images. The docker-compose.yml at the monorepo root defines a multi-service development environment that starts both services with a single docker compose up --build command. The compose file maps the appropriate ports (3000 for the frontend, 4000 for the backend) and sets environment variables via .env files.

10.2 CI/CD Pipeline

A GitHub Actions CI/CD pipeline is defined in .github/workflows/. The pipeline triggers on push to the main branch and performs the following stages: code checkout, Node.js environment setup, dependency installation, TypeScript compilation, unit test execution (Jest), end-to-end test execution (Playwright), Docker image build for both services, and deployment to the production hosting environment. Secrets such as JWT signing keys, database credentials, and SMTP credentials are stored as GitHub repository secrets and injected as environment variables during the deployment stage.

10.3 Production Hosting

ShopSmart is deployed to production at the taj.works domain, with three subdomains corresponding to the three deployed services: shopsmart.taj.works (Next.js frontend), shopsmart-server.taj.works (Express.js backend API), and shopsmart-swagger.taj.works (Swagger documentation). HTTPS is enforced for all three services. Cookie security is configured with secure: true, sameSite: 'none', and a shared COOKIE_DOMAIN of .taj.works to enable cross-subdomain cookie sharing between the frontend and API.

10.4 AWS Infrastructure (Terraform)

An optional AWS infrastructure definition is provided in the terraform/ directory. The Terraform configuration provisions: a VPC with public subnets across two availability zones; an Application Load Balancer (ALB) listening on ports 80 and 8080 for the API and client services respectively; an ECS Fargate cluster with services for both containers; ECR repositories for storing the Docker images; an IAM task execution role with appropriate permissions; a private S3 bucket with access from the ECS task role for asset storage; and an optional EC2 instance with SSM access for administration. PostgreSQL is not provisioned by this stack and is expected to be supplied via a managed database service such as AWS RDS.

11. Features & Functionalities

11.1 Customer Features

- Product catalogue browsing with filtering by category, price range, and availability, and sorting by price, popularity, and recency.
- Full-text product search with debounced input.
- Shopping cart management — add items, update quantities, remove items, and persist cart across sessions.
- Wishlist management for saving products for future consideration.
- Secure checkout with order placement and confirmation.
- Order history and order detail views with real-time status tracking.
- Product review submission with rating and text content.
- Profile management — update personal information, change password, manage addresses.
- Email verification flow on registration, with resend capability.
- Forgot password and self-service password reset via email token.

11.2 Administration Features

- KPI dashboard with key metrics — total users, products, orders, and revenue — with trend indicators.
- Product CRUD — create, update, archive, and restore products with image upload and inventory tracking.
- Category management — create and manage product categories.
- Inventory management — track stock quantities and in-stock status.
- Order management — view all orders, update status (processing, shipped, delivered, cancelled, refunded).
- Customer management — view customer accounts, update status, and view order history.
- Review moderation — approve, reject, or delete customer reviews.
- Coupon and discount management — create PERCENT and FIXED discount coupons with validity windows.
- Sales and revenue reports with date range filtering and chart visualisations.
- Audit log viewer — review all system actions with timestamps and user attribution.

11.3 Super Administration Features

- Admin account lifecycle management — create new admin users, update their profiles, and delete accounts.
- Role assignment — assign and modify user roles across the platform.
- System configuration management — update platform-wide settings stored in SystemConfig.
- Payment gateway configuration — manage gateway settings and credentials.
- Shipping provider management — configure shipping strategies (free, flat rate, threshold-based).

- Feature flag management — enable or disable platform features at runtime without redeployment.
- Full analytics dashboard — platform-wide order volumes, revenue trends, user growth, and product performance.
- Complete override access — super admins have access to all admin-level functions.

12. Testing & Validation

12.1 End-to-End Testing (Playwright)

End-to-end tests are located in the e2e/ directory and executed using Playwright. The test suite validates critical application flows in a real browser environment, covering: the home page load and rendering, the API health endpoint response, and core navigation flows. Playwright's cross-browser support ensures consistent behaviour across Chromium, Firefox, and WebKit.

The E2E tests are integrated into the GitHub Actions CI/CD pipeline and run as a quality gate before each production deployment. Test execution against the production URL can be configured via environment variables in the Playwright configuration.

12.2 Unit Testing (Jest)

Backend unit tests are located in src/**/__tests__/ directories and executed with Jest and ts-jest for TypeScript support. The application of the Dependency Inversion Principle throughout the backend makes unit testing highly effective — service classes accept all dependencies as constructor parameters typed against interfaces, enabling complete mock injection.

A representative unit test for AuthService.register demonstrates the approach: a mock IUserRepository is created with jest.fn() for findByEmail and create, a mock IEmailService is created with jest.fn() for sendVerification, and a mock ITokenService is created for token generation. The service is instantiated with these mocks, the register method is invoked, and assertions verify that the correct sequence of calls was made with the correct arguments — all without touching a real database or SMTP server.

12.3 Manual API Testing (Swagger)

The interactive Swagger UI at shopsmart-swagger.taj.works/api-docs serves as both documentation and a manual testing environment. All endpoints are testable directly from the browser using the Authorize button to provide a Bearer token obtained from the login endpoint. This facilitates both developer testing during development and stakeholder demonstration of API functionality.

12.4 Security Validation

Security measures were validated through the following checks: httpOnly cookie isolation was verified by confirming that document.cookie in the browser console does not expose accessToken or refreshToken; CORS enforcement was tested by confirming rejection of requests from origins not matching FRONTEND_URL; role-based access control was tested by attempting to access admin endpoints with customer tokens and super-admin endpoints with admin tokens, confirming correct 403 responses; and login lockout was validated by submitting more than the configured MAX_FAILED_LOGINS attempts and confirming the account lock response.

13. Results & Screenshots

ShopSmart is fully deployed and operational at shopsmart.taj.works. The following describes the key screens and their implemented functionality:

13.1 Customer Storefront

The homepage presents a curated product grid with category navigation, featured products, and deals. The product listing page supports live filtering and sorting without page reloads. Individual product pages display full product details, pricing, stock availability, reviews, and an add-to-cart button. The cart sidebar updates in real time as items are added or modified. The checkout flow captures shipping address and confirms order placement.

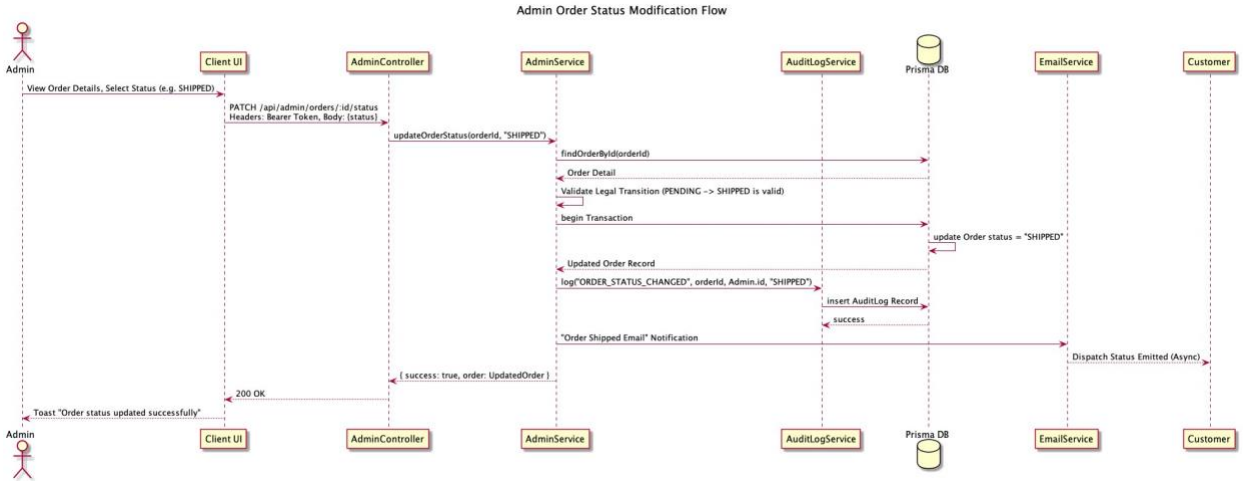

Figure 12: Admin Order Status Modification Flow — Sequence Diagram

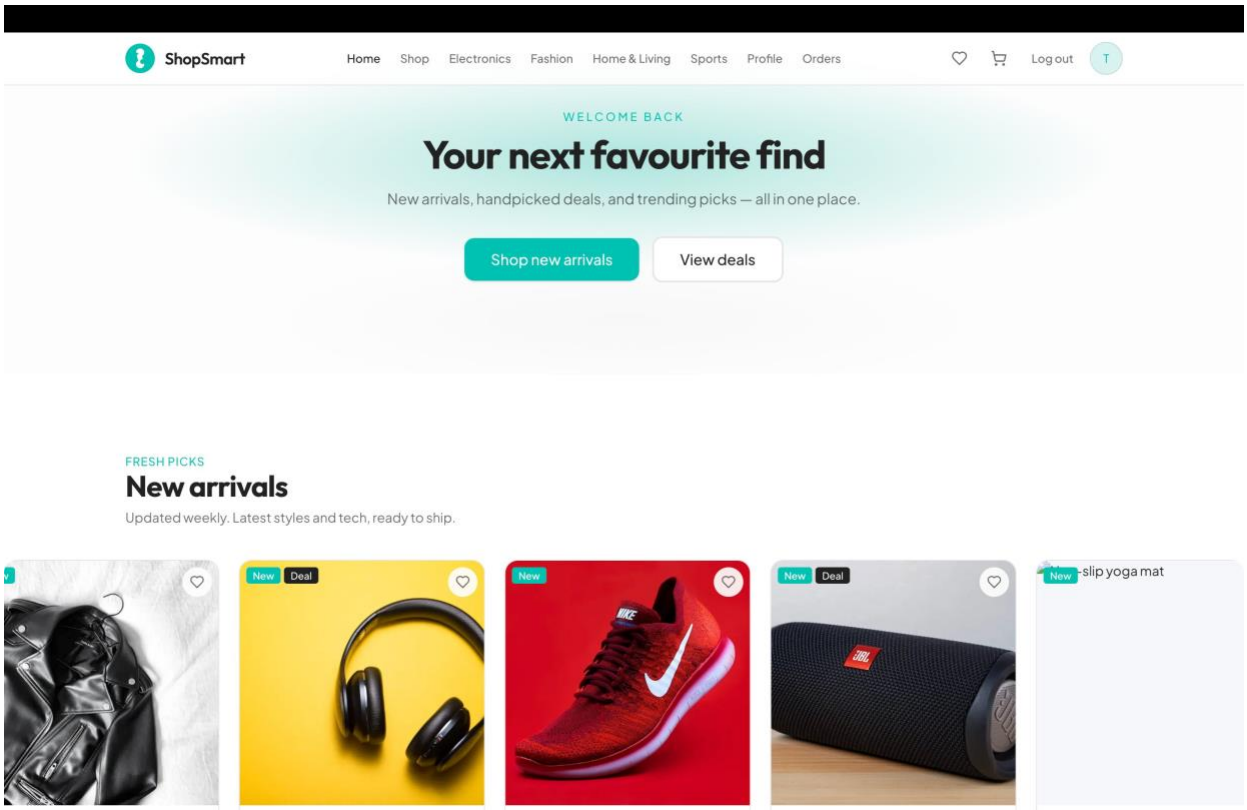

Figure 13: ShopSmart Customer Storefront — Homepage with New Arrivals

13.2 Admin Dashboard

The admin dashboard presents four KPI cards (total users, products, orders, and revenue) with percentage change indicators. A sales chart rendered using Recharts shows daily revenue over the selected period. The products table supports inline editing and deletion. The orders table allows bulk status updates. The coupons section provides a form for creating new discount codes with type (PERCENT/FIXED), value, and validity period.

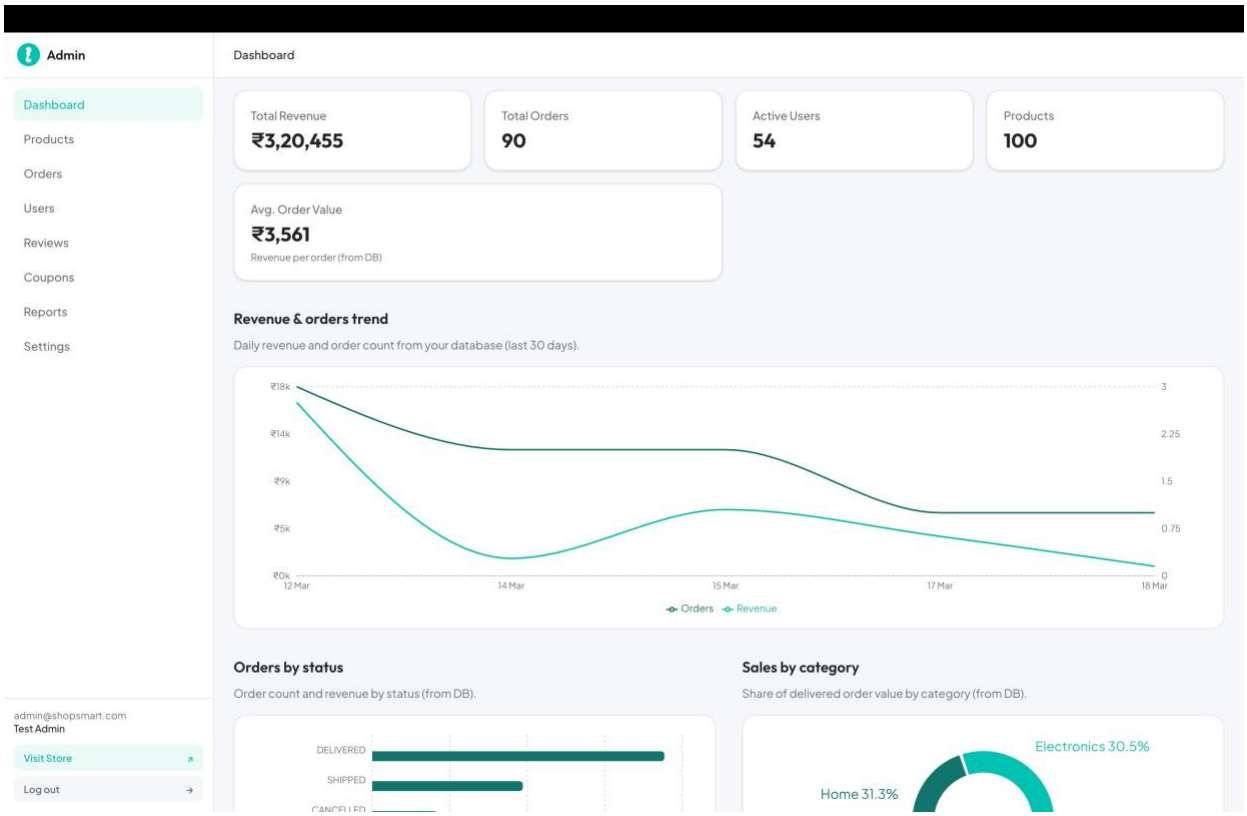

Figure 14: Admin Dashboard — KPI Cards, Revenue & Orders Trend (Recharts)

13.3 Super Admin Panel

The super admin panel provides an admins management table with create and delete actions. The system configuration section displays all editable SystemConfig values in a form. The feature flags section lists all platform features as toggles. The analytics section shows platform-wide metrics with chart visualisations.

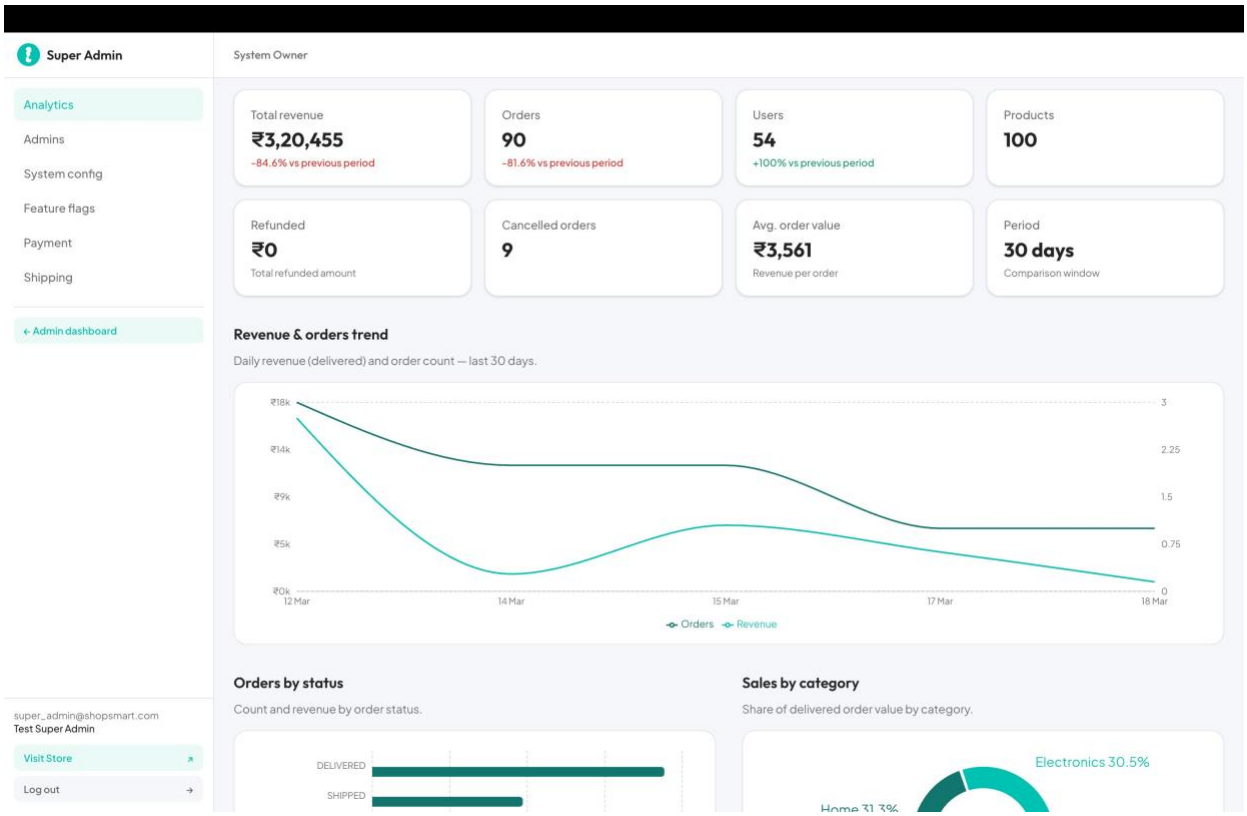

Figure 15: Super Admin Panel — Analytics, System Config, Feature Flags

13.4 Swagger Documentation

The Swagger UI at shopsmart-swagger.taj.works/api-docs presents all API modules in expandable sections. Each endpoint shows the HTTP method, path, description, request body schema, query parameters, and all possible response schemas including error codes. The Authorize button enables authenticated testing directly in the browser.

ShopSmart API 1.0.0 OAS 3.0

Architecture diagram

→ Open architecture diagram

Monorepo structure, request lifecycle, RBAC, and tech stack.

Overview

This is the **ShopSmart API**: the backend for the ShopSmart eCommerce app. It is a REST API: you send HTTP requests (GET, POST, PATCH, DELETE) to the URLs below; responses are JSON.

Live deployment:

- **Frontend:** `https://shopsmart.taj.works/`
- **Backend API:** `https://shopsmart-server.taj.works/`
- **Swagger / API docs:** `https://shopsmart-swagger.taj.works/api-docs`

What each section does:

- **Health** — Check if the API is running (no login required).
- **Auth** — Register, log in, refresh token, log out. The server sets httpOnly cookies (access + refresh) for browser clients. For **Swagger / API testing**, the response body includes `accessToken` : use it in the **Authorize** button as `Bearer <accessToken>` .
- **Users** — Get or update the current user profile. **Requires auth.** Admins can list and manage users (Admin / Super Admin).
- **Categories** — List product categories (public). Create/update categories require **Admin** or **Super Admin**.
- **Products** — List and get products (public). Create/update/delete require **Admin** or **Super Admin**.
- **Cart** — Requires **authenticated user**. Add, update, or remove cart items.
- **Orders** — Create and list orders require **authenticated user**. Update order status requires **Admin** or **Super Admin**.
- **Reviews** — Create review requires **authenticated user**. Delete/update status require **Admin** or **Super Admin**.
- **Admin** — Dashboard, stats, revenue, logs, coupons, reports. **Admin** or **Super Admin** only.
- **Super Admin** — System config, create/delete admins, assign roles. **Super Admin** only.

Endpoints that require authentication show a lock icon. Use **Authorize** and enter `Bearer <your-access-token>` (paste the token from login/register/refresh response). The server accepts either the `Authorization: Bearer` header or the access token cookie.

Authentication

For Swagger "Try it out":

1. Call **POST /api/auth/login** with `email` and `password` (e.g. `customer@shopsmart.com` / `Customer1!`).
2. Copy `accessToken` from the response body.
3. Click **Authorize** (top right), enter: `Bearer <paste-token-here>` , then **Authorize**.
4. Protected endpoints will now send the token automatically.

For browser / cookie-based clients:

- Login, register, and refresh set **httpOnly cookies** (`accessToken` , `refreshToken`). Use `credentials: "include"` (fetch) or `withCredentials: true` (Axios). No need to send the token in the header.

Figure 16: Swagger UI (OpenAPI 3.0) at shopsmart-swagger.taj.works/api-docs

Servers
https://shopsmart-server.taj.works - Production (API)

Authorize

Health Health check

- GET `/api/health` Health check

Auth Authentication and account

- POST `/api/auth/register` Register a new user
- POST `/api/auth/login` Login
- POST `/api/auth/refresh` Refresh access token
- POST `/api/auth/logout` Logout
- GET `/api/auth/me` Get current user
- POST `/api/auth/verify-email` Verify email
- POST `/api/auth/forgot-password` Forgot password
- POST `/api/auth/reset-password` Reset password

Users User management

- GET `/api/users` List users
- GET `/api/users/{id}` Get user by ID

14. Challenges Faced & Solutions

Challenge	Solution
Cross-origin cookie transmission between shopsmart.taj.works and shopsmart-server.taj.works required SameSite=None and Secure flags, which had to be carefully managed across development (lax) and production (none + secure) environments.	Introduced environment-based cookie configuration using COOKIE_SAME_SITE and COOKIE_SECURE environment variables. Shared COOKIE_DOMAIN=.taj.works enables subdomain cookie sharing in production while localhost:3000 development uses SameSite=lax without the Secure flag.
Coordinating a five-member team across frontend, backend, design pattern implementation, UML documentation, and business logic without creating merge conflicts or architectural inconsistencies.	The team adopted a strict module ownership model with clear boundaries documented in the README. Feature branches were named by module and contributor, and pull requests required review before merging to main. The container.ts composition root served as the integration point.
Prisma Decimal type leaking to API responses caused JavaScript floating-point inconsistencies and exposed internal model details.	The Adapter pattern was applied uniformly. All Prisma models are transformed to DTOs through dedicated Adapter classes (UserAdapter, ProductAdapter, OrderAdapter) before leaving the service layer, ensuring Decimal conversion and sensitive field exclusion.
The Observer pattern's EventBus required careful error isolation to prevent side-effect failures from propagating to the main request flow and causing request failures.	Promise.allSettled() was used in EventBus.emit() to run all observer handlers concurrently without allowing any individual observer failure to reject the parent promise. Errors are caught per-observer and logged via LoggerService.
TypeScript strict mode required explicit typing for all constructor-injected dependencies, creating verbose type declarations.	Interface segregation was applied to keep interfaces focused and minimal. TypeScript's ReturnType<> and Extract<> utilities reduced boilerplate in factory and configuration code.
Terraform AWS deployment required careful configuration of ECS task definitions, ALB target groups, and health check paths to avoid deployment failures during container health check grace periods.	The API health check endpoint at /api/health was designed to respond within milliseconds. ALB health check configuration uses a 30-second interval with a 5-second timeout and 2 healthy threshold, allowing new container instances sufficient warm-up time.

15. Future Enhancements

The ShopSmart codebase has been architecturally designed with explicit extension points for future development:

- Real Payment Gateway Integration: The IPaymentGateway interface and StripeAdapter stub provide a ready-made integration point. Completing the Stripe integration requires only implementing the charge() and refund() methods in StripeAdapter and configuring the Stripe secret key.
- OAuth Authentication: An OAuthJwtTokenService implementing ITokenStrategy is already referenced in the codebase. Setting AUTH_PROVIDER=oauth in the environment will swap the token service without modifying AuthService.
- SendGrid Email Provider: A SendGridStrategy implementing IEmailProviderStrategy can be activated by setting EMAIL_PROVIDER=sendgrid, providing a higher-deliverability email provider for transactional emails.
- SMS and Push Notifications: The INotificationService interface and NotificationObserver are designed to support additional notification channels. A new TwilioSmsNotificationService can be implemented and registered in container.ts without modifying any existing code.
- Product Recommendation Engine: The product browsing and order history data collected by the platform provides a foundation for a collaborative filtering recommendation engine. An IRecommendationService interface can be defined and integrated into the product service layer.
- Elasticsearch Integration: As the product catalogue grows, full-text search backed by Elasticsearch can replace or supplement the current database-level filtering by implementing an IProductSearchRepository and injecting it into ProductService.
- Real-Time Order Tracking: WebSocket support can be added to the Express server using socket.io, with the OrderStatusChangedEvent triggering real-time push updates to the client application.
- Mobile Application: A React Native mobile client can consume the existing REST API with no backend changes, leveraging the same OpenAPI specification for type-safe API client generation.
- Advanced Analytics: The existing AuditLog and Order data can feed a dedicated analytics pipeline using a time-series database for richer business intelligence and reporting.

16. Team Contributions Overview

This section documents the specific contributions of each team member to the ShopSmart project.

Shaik Tajuddin (GitHub: Taj-2005) — Project Founder & Team Lead

Shaik Tajuddin founded the ShopSmart project and led the team from inception through production deployment. He owned the end-to-end system design including the overall monorepo architecture, module decomposition, API specification design, and full-stack foundation. He implemented the initial frontend application structure using Next.js 15 App Router, the backend Express.js server with its routing, middleware, and configuration layers, the Prisma database schema, and the Swagger/OpenAPI 3.0 documentation. He was responsible for the production deployment to the taj.works domain, including HTTPS configuration, subdomain routing, and cookie cross-origin configuration. He also established the Git workflow, code standards, and contribution guidelines followed by the team.

Pranavi (GitHub: pranavi015) — SOLID Principles Refactoring

Pranavi led the comprehensive refactoring of the backend codebase to comply with all five SOLID principles. Working from the SOLID principles guide, she systematically extracted responsibilities from monolithic route handlers and service files, creating dedicated service classes, interface definitions, and a structured module hierarchy. She designed the modular class structures across the interfaces/, services/, and modules/ directories, established the four-layer module pattern (routes, controller, service, validation), and improved overall code maintainability and testability. Her refactoring work created the foundation on which design patterns could be cleanly applied.

Omkar Hadole (GitHub: omkar-hadole) — Design Patterns & Business Logic

Omkar identified the architectural gaps in the codebase and implemented core design patterns and business logic aligned with the refactored SOLID-compliant architecture. He implemented the Repository pattern with IRepository interfaces and PrismaRepository implementations, the Factory pattern with AppErrorFactory and ApiResponseFactory, the Observer pattern with EventBus and AuditLogObserver, and the Strategy pattern with discount and shipping strategy families. He also implemented the core business logic for order management, including pricing calculation, stock validation, order status transitions, and coupon validation, leveraging the proper design patterns throughout.

Praneeth (GitHub: iampraneeth1116) — UML Diagrams & Documentation

Praneeth was responsible for all UML artefacts and technical documentation. He developed the complete set of PlantUML diagrams — Entity-Relationship diagram, Backend Services Class diagram, Component/System Architecture diagram, Deployment diagram, and Server Package Structure diagram — stored in the docs/uml/ directory and referenced in the README. He also contributed to architecture design discussions, prepared detailed project documentation, and created the architecture.html reference document that describes the full system architecture in a browsable format. His documentation work was instrumental in aligning the team's understanding of the system.

Ashrith (GitHub: ashrith-07) — Business Logic Implementation

Ashrith implemented business logic modules adhering to the SOLID-based class structures defined during the refactoring phase and applying appropriate design patterns throughout. His contributions include service implementations for the admin module (dashboard KPIs, report generation, coupon management, review moderation), the super-admin module (admin lifecycle management, feature flags, system configuration), and supporting business logic for the authentication flows. His implementations consistently followed the constructor injection pattern, depended on interfaces rather than concrete classes, and used the AppErrorFactory and ApiResponseFactory for consistent error and response handling.

17. Live Deployments

ShopSmart is live in production. All three services are deployed under the taj.works domain with HTTPS enforced and persistent uptime.

Service	URL	Description
Frontend (Next.js)	https://shopsmart.taj.works/	Customer storefront, admin and super-admin panels
Backend API (Express)	https://shopsmart-server.taj.works/	REST API base URL
Swagger Documentation	https://shopsmart-swagger.taj.works/api-docs	Interactive OpenAPI 3.0 API documentation
API Health Check	https://shopsmart-server.taj.works/api/health	System health endpoint

Domain: taj.works — all services served over HTTPS with TLS certificates. Cookie configuration uses COOKIE_DOMAIN=.taj.works for subdomain sharing, COOKIE_SECURE=true, and COOKIE_SAME_SITE=none.

18. Conclusion

ShopSmart successfully demonstrates that software engineering principles and design patterns are not abstract theoretical constructs, but practical tools that solve real architectural problems in real codebases. The project began with an honest assessment of existing violations and proceeded systematically to address each one, producing a codebase that is genuinely more testable, more maintainable, and more extensible than its starting state.

The application of all five SOLID principles transformed a collection of tightly-coupled route handlers into a cleanly layered architecture where each class has a single reason to change, new features can be added without modifying existing code, subtypes are genuinely substitutable, interfaces are focused on specific consumer needs, and business logic depends on abstractions rather than infrastructure concretions.

The ten design patterns implemented each address a specific, observable problem: the Singleton prevents resource duplication; the Observer decouples side effects from primary flows; the Factory eliminates scattered new AppError() calls; the Template Method removes repeated controller boilerplate; the Strategy makes pricing and notification channels swappable; the Adapter prevents internal Prisma model leakage to the API layer; the Repository makes the entire service layer testable without a database; the Chain of Responsibility enforces consistent middleware ordering; the Decorator adds cross-cutting concerns without modifying service classes; and the Command ensures every mutation generates an audit trail.

Beyond the engineering aspects, ShopSmart delivers a genuinely functional production application — deployed at shopsmart.taj.works — with a complete customer storefront, a feature-rich administration panel, and a comprehensive super-administration system. The inclusion of Docker containerisation, CI/CD via GitHub Actions, E2E testing with Playwright, unit testing with Jest, and optional AWS Terraform infrastructure demonstrates the team's understanding of the full software development lifecycle, from architecture to production operations.

This project reflects the course's central thesis: that thinking like a software engineer means prioritising design clarity, maintainability, and testability — not just functionality. The ShopSmart codebase is a working, deployed demonstration of that principle in practice.

19. References

10. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
11. Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
12. Martin, R. C. (2002). Agile Software Development, Principles, Patterns, and Practices. Prentice Hall.
13. Freeman, E., & Robson, E. (2020). Head First Design Patterns (2nd ed.). O'Reilly Media.
14. Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
15. Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
16. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
17. Humble, J., & Farley, D. (2010). Continuous Delivery. Addison-Wesley.
18. Google SRE Team. (2016). Site Reliability Engineering. O'Reilly Media.
19. Next.js Documentation. (2024). App Router. https://nextjs.org/docs
20. Express.js Documentation. (2024). Express 4.x API Reference. https://expressjs.com/en/api.html
21. Prisma Documentation. (2024). Prisma ORM. https://www.prisma.io/docs
22. OpenAPI Initiative. (2021). OpenAPI Specification 3.0.3. https://swagger.io/specification/
23. TypeScript Documentation. (2024). TypeScript Handbook. https://www.typescriptlang.org/docs/handbook/
24. Playwright Documentation. (2024). Playwright for TypeScript. https://playwright.dev/docs/intro
25. Terraform Documentation. (2024). AWS Provider. https://registry.terraform.io/providers/hashicorp/aws/latest/docs
26. ShopSmart GitHub Repository. (2026). https://github.com/Taj-2005/shop_smart
27. ShopSmart Live Application. (2026). https://shopsmart.taj.works
28. ShopSmart API Documentation. (2026). https://shopsmart-swagger.taj.works/api-docs